

Commenced Publication in 1973

Founding and Former Series Editors:

Gerhard Goos, Juris Hartmanis, and Jan van Leeuwen

Editorial Board

David Hutchison

Lancaster University, Lancaster, UK

Takeo Kanade

Carnegie Mellon University, Pittsburgh, PA, USA

Josef Kittler

University of Surrey, Guildford, UK

Jon M. Kleinberg

Cornell University, Ithaca, NY, USA

Friedemann Mattern

ETH Zurich, Zürich, Switzerland

John C. Mitchell

Stanford University, Stanford, CA, USA

Moni Naor

Weizmann Institute of Science, Rehovot, Israel

C. Pandu Rangan

Indian Institute of Technology, Madras, India

Bernhard Steffen

TU Dortmund University, Dortmund, Germany

Demetri Terzopoulos

University of California, Los Angeles, CA, USA

Doug Tygar

University of California, Berkeley, CA, USA

Gerhard Weikum

Max Planck Institute for Informatics, Saarbrücken, Germany

More information about this series at <http://www.springer.com/series/7407>

Nadia Creignou · Daniel Le Berre (Eds.)

Theory and Applications of Satisfiability Testing – SAT 2016

19th International Conference
Bordeaux, France, July 5–8, 2016
Proceedings

Editors

Nadia Creignou
Aix-Marseille Université
Marseille
France

Daniel Le Berre
Université d'Artois
Lens
France

ISSN 0302-9743 ISSN 1611-3349 (electronic)
Lecture Notes in Computer Science
ISBN 978-3-319-40969-6 ISBN 978-3-319-40970-2 (eBook)
DOI 10.1007/978-3-319-40970-2

Library of Congress Control Number: 2016941614

LNCS Sublibrary: SL1 – Theoretical Computer Science and General Issues

© Springer International Publishing Switzerland 2016

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made.

Printed on acid-free paper

This Springer imprint is published by Springer Nature
The registered company is Springer International Publishing AG Switzerland

Preface

This volume contains the papers presented at the 19th International Conference on Theory and Applications of Satisfiability Testing (SAT 2016) held during July 5–8, 2016, in Bordeaux, France. SAT 2016 was hosted by the Computer Science Laboratory of Bordeaux (LaBRI).

The International Conference on Theory and Applications of Satisfiability Testing (SAT) is the premier annual meeting for researchers focusing on the theory and applications of the propositional satisfiability problem, broadly construed. Aside from plain propositional satisfiability, the scope of the meeting includes Boolean optimization (including MaxSAT and pseudo-Boolean (PB) constraints), quantified Boolean formulas (QBF), satisfiability modulo theories (SMT), and constraint programming (CP) for problems with clear connections to Boolean-level reasoning. Many hard combinatorial problems can be tackled using SAT-based techniques, including problems that arise in formal verification, artificial intelligence, operations research, computational biology, cryptology, data mining, machine learning, mathematics, etc. Indeed, the theoretical and practical advances in SAT research over the past 20 years have contributed to making SAT technology an indispensable tool in a variety of domains.

SAT 2016 welcomed scientific contributions addressing different aspects of SAT interpreted in a broad sense, including (but not restricted to) theoretical advances (including exact algorithms, proof complexity, and other complexity issues), practical search algorithms, knowledge compilation, implementation-level details of SAT solvers and SAT-based systems, problem encodings and reformulations, applications (including both novel applications domains and improvements to existing approaches), as well as case studies and reports on findings based on rigorous experimentation.

A total of 70 papers were submitted this year distributed into 48 long papers, 13 short papers, and nine tool papers. The papers were reviewed by the Program Committee (33 members), with the help of 65 additional reviewers. Only one regular paper was found by the Program Committee to be out of the scope for the conference. Each of the remaining submissions was reviewed by at least three different reviewers. A rebuttal period allowed the authors to provide a feedback to the reviewers. After that, the discussion among the Program Committee took place. External reviewers supporting the Program Committee were also invited to participate directly in the discussions for the papers they reviewed. This year, the authors received a meta-review, summarizing the discussion that occurred after the rebuttal and the reasons of the final recommendation. The final recommendation was to accept 31 submissions (22 long, four short, and five tool papers) and to accept conditionally five additional papers. The latter (four long and one short) eventually satisfied the conditions for acceptance.

In addition to presentations on the accepted papers, the scientific program of SAT 2016 included three invited talks:

- Phokion Kolaitis (University of California Santa Cruz, IBM, USA) “Coping with Inconsistent Databases: Semantics, Algorithms, and Complexity”
- David Monniaux (VERIMAG University of Grenoble, CNRS, France) “Satisfiability Testing, a Disruptive Technology in Program Verification”
- Torsten Schaub (University of Potsdam, Germany, EurAI sponsored) “From SAT to ASP and Back!?”

As in previous years, SAT 2016 hosted various associated events, including four workshops on July 4:

- 6th International Workshop on the Cross-Fertilization Between CSP and SAT (CSPSAT 2016) organized by Yael Ben-Haim, Valentin Mayer-Eichberger, and Yehuda Naveh
- “Graph Structure and Satisfiability Testing” organized by Simone Bova and Stefan Mengel
- 7th Pragmatics of SAT International Workshop (PoS 2016) organized by Olivier Roussel and Allen Van Gelder
- 4th International Workshop on Quantified Boolean Formulas (QBF 2016) organized by Florian Lonsing and Martina Seidl

There were also four competitive events, which ran before the conference and whose results were disclosed during the conference:

- MAXSAT evaluation organized by Josep Argelich, Chu Min Li, Felip Manyà and Jordi Planes
- PB competition organized by Olivier Roussel
- QBF evaluation organized by Luca Pulina
- SAT competition organized by Marijn Heule, Matti Jarvisalo, and Tomas Baylo

Moreover, this year a full day of tutorials — “How to Solve My Problem with SAT?” — was organized right after the conference, on July 9.

March 2016 was a terrible month for the SAT community. On March 12, Helmut Veith, our esteemed colleague from TU Vienna, passed away at the age of 45. His work on counter example guided abstraction refinement is widely used in the SAT community, especially in recent years to tackle QBF problems: A specific session on that topic was organized during the conference. On March 13, Hilary Putnam, one of the authors of the seminal “Davis and Putnam” procedure, central in current SAT research, passed away at the age of 90. The first session on SAT solving was dedicated to his memory. Our thoughts are with their families during this difficult time.

We would like to thank everyone who contributed to making SAT 2016 a success. First and foremost we would like to thank the members of the Program Committee and the additional reviewers for their careful and thorough work, without which it would not have been possible for us to put together such an outstanding conference. We also wish to thank all the authors who submitted their work for our consideration. We thank the SAT Association chair Armin Biere, vice chair John Franco, and treasurer Hans Kleine Büning for their help and advice in organizational matters. The EasyChair conference systems provided invaluable assistance in coordinating the submission and review process, in organizing the program, as well as in the assembly of these

proceedings. We also thank the local organization team for their efforts with practical aspects of local organization.

Finally, we gratefully thank the University of Bordeaux, Bordeaux INP, the Computer Science Laboratory of Bordeaux (LaBRI), the GIS Albatros (Bordeaux), the CNRS, the Laboratory of Fundamental Computer Science of Marseilles (LIF), the Lens Computer Science Research Laboratory (CRIL), the European Association for Artificial Intelligence (EurAI), the SAT association, the French-Speaking Constraints Association (AFPC), Intel, RATP and Safe-River for financial and organizational support for SAT 2016.

April 2016

Daniel Le Berre
Nadia Creignou

Organization

Program Committee

Fahiem Bacchus	University of Toronto, Canada
Yael Ben-Haim	IBM Research, Israel
Olaf Beyersdorff	University of Leeds, UK
Armin Biere	Johannes Kepler University, Austria
Nikolaj Björner	Microsoft Research, USA
Maria Luisa Bonet	Universitat Politècnica de Catalunya, Spain
Sam Buss	UCSD, USA
Nadia Creignou	Aix-Marseille Université, LIF-CNRS, France
Uwe Egly	TU Wien, Austria
John Franco	University of Cincinnati, USA
Djamal Habet	Aix-Marseille Université, LSIS-CNRS, France
Marijn Heule	The University of Texas at Austin, USA
Holger Hoos	University of British Columbia, Canada
Frank Hutter	University of Freiburg, Germany
Mikolas Janota	Microsoft Research, UK
Matti Järvisalo	University of Helsinki, Finland
Hans Kleine Büning	University of Paderborn, Germany
Daniel Le Berre	Université d'Artois, CRIL-CNRS, France
Ines Lynce	INESC-ID/IST, University of Lisbon, Portugal
Marco Maratea	DIBRIS, University of Genoa, Italy
Joao Marques-Silva	Faculty of Science, University of Lisbon, Portugal
Stefan Mengel	CRIL-CNRS, France
Alexander Nadel	Intel, Israel
Nina Narodytska	Samsung Research America, USA
Jakob Nordström	KTH Royal Institute of Technology, Sweden
Albert Oliveras	Technical University of Catalonia, Spain
Roberto Sebastiani	DISI, University of Trento, Italy
Martina Seidl	Johannes Kepler University Linz, Austria
Yuping Shen	Institute of Logic and Cognition, Sun Yat-sen University, China
Laurent Simon	Labri, Bordeaux Institute of Technology, France
Takehide Soh	Information Science and Technology Center, Kobe University, Japan
Stefan Szeider	TU Wien, Austria
Allen Van Gelder	University of California, Santa Cruz, USA

Additional Reviewers

Abramé, André
 Aleksandrowicz, Gadi
 Audemard, Gilles
 Banbara, Mutsunori
 Baud-Berthier, Guillaume
 Bayless, Sam
 Berkholz, Christoph
 Blinkhorn, Joshua
 Bofill, Miquel
 Bonacina, Ilario
 Cameron, Chris
 Cao, Weiwei
 Chew, Leroy
 de Rezende, Susanna F.
 Demri, Stéphane
 Dodaro, Carmine
 Eggensperger, Katharina
 Elffers, Jan
 Falkner, Stefan
 Feng, Shiguang
 Fichte, Johannes Klaus
 Fröhlich, Andreas
 Galesi, Nicola
 Ganesh, Vijay
 Gange, Graeme
 Gaspers, Serge
 Griggio, Alberto
 Guthmann, Ofer
 Hermo, Montserrat
 Hinde, Luke
 Ibanez-Garcia, Yazmin Angelica
 Ignatiev, Alexey
 Ivrii, Alexander

Kolaitis, Phokion
 Kotthoff, Lars
 Kovásznaï, Gergely
 Lauria, Massimo
 Lettmann, Theodor
 Lindauer, Marius
 Lonsing, Florian
 Malod, Guillaume
 Manquinho, Vasco
 Marquis, Pierre
 Martins, Ruben
 Meel, Kuldeep S.
 Miksa, Mladen
 Nabeshima, Hidetomo
 Naveh, Yehuda
 Neves, Miguel
 Oetsch, Johannes
 Panagiotou, Konstantinos
 Pich, Ján
 Prcovic, Nicolas
 Preiner, Mathias
 Rozier, Kristin Yvonne
 Ryvchin, Vadim
 Slivovsky, Friedrich
 Strichman, Ofer
 Trentin, Patrick
 Tveretina, Olga
 Vinyals, Marc
 Wang, Lingli
 Widl, Magdalena
 Wintersteiger, Christoph M.
 Yue, Weiya

Invited Talks

Coping with Inconsistent Databases: Semantics, Algorithms, and Complexity

Phokion G. Kolaitis^{1,2}

¹ University of California Santa Cruz, Santa Cruz, USA

² IBM Research – Almaden, San Jose, USA
kolaitis@cs.ucsc.edu

Abstract. Managing inconsistency in databases is a long-standing challenge. The framework of database repairs provides a principled approach towards coping with inconsistency in databases. Intuitively, a repair of an inconsistent database is a consistent database that differs from the given inconsistent database in a minimal way. Repair checking and consistent query answering are two fundamental algorithmic problems arising in this context. The first of these two problems asks whether, given two databases, one is a repair of the other. The second asks whether, a query is true on every repair of a given inconsistent database.

The aim of this talk is to give an overview of a body of results in this area with emphasis on the computational complexity of repair checking and consistent query answering, including the quest for dichotomy theorems. In addition to presenting open problems, the last part of the talk will include a discussion of the potential use of solvers in developing practical systems for consistent query answering.

Keywords: Inconsistent databases · Database dependencies · Database repairs · Consistent query answering · Computational complexity

References

1. Afrati, F.N., Kolaitis, P.G.: Repair checking in inconsistent databases: algorithms and complexity. In: 12th International Conference on Database Theory. ICDT 2009, St. Petersburg, Russia, March 23–25, 2009, Proceedings, pp. 31–41 (2009)
2. Arenas, M., Bertossi, L.E., Chomicki, J.: Consistent query answers in inconsistent databases. In: Proceedings of the Eighteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, May 31 – June 2, 1999, Philadelphia, Pennsylvania, USA, pp. 68–79 (1999)
3. Arming, S., Pichler, R., Sallinger, E.: Complexity of repair checking and consistent query answering. In: 19th International Conference on Database Theory. ICDT 2016, Bordeaux, France, March 15–18, 2016, pp. 21:1–21:18 (2016)
4. Bertossi, L.E.: Database Repairing and Consistent Query Answering. Synthesis Lectures on Data Management. Morgan & Claypool Publishers (2011)
5. ten Cate, B., Fontaine, G., Kolaitis, P.G.: On the data complexity of consistent query answering. *Theory Comput. Syst.* **57**(4), 843–891 (2015)

6. ten Cate, B., Halpert, R.L., Kolaitis, P.G.: Exchange-repairs: managing inconsistency in data exchange. In: Kontchakov, R., Mugnier, M.L. (eds.) RR 2014. LNCS, vol. 8741, pp. 140–156. Springer, Switzerland (2014)
7. ten Cate, B., Halpert, R.L., Kolaitis, P.G.: Practical query answering in data exchange under inconsistency-tolerant semantics. In: Proceedings of the 19th International Conference on Extending Database Technology. EDBT 2016, Bordeaux, France, March 15–16, 2016, pp. 233–244 (2016)
8. Chomicki, J.: Consistent query answering: five easy pieces. In: 11th International Conference on Database Theory - ICDT 2007, Barcelona, Spain, January 10–12, 2007, Proceedings, pp. 1–17 (2007)
9. Chomicki, J., Marcinkowski, J.: Minimal-change integrity maintenance using tuple deletions. *Inf. Comput.* **197**(1–2), 90–121 (2005)
10. Chomicki, J., Marcinkowski, J., Staworko, S.: Hippo: a system for computing consistent answers to a class of SQL queries. In: Advances in Database Technology - EDBT 2004, 9th International Conference on Extending Database Technology, Heraklion, Crete, Greece, March 14–18, 2004, Proceedings, pp. 841–844 (2004)
11. Fagin, R., Kimelfeld, B., Kolaitis, P.G.: Dichotomies in the complexity of preferred repairs. In: Proceedings of the 34th ACM Symposium on Principles of Database Systems. PODS 2015, Melbourne, Victoria, Australia, May 31 – June 4, 2015, pp. 3–15 (2015)
12. Fontaine, G.: Why is it hard to obtain a dichotomy for consistent query answering? *ACM Trans. Comput. Log.* **16**(1), 7:1–7:24 (2015)
13. Fuxman, A., Fazli, E., Miller, R.J.: ConQuer: Efficient management of inconsistent databases. In: Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14–16, 2005, pp. 155–166 (2005)
14. Fuxman, A., Fuxman, D., Miller, R.J.: ConQuer: a system for efficient querying over inconsistent databases. In: Proceedings of the 31st International Conference on Very Large Data Bases, Trondheim, Norway, August 30 – September 2, 2005, pp. 1354–1357 (2005)
15. Kolaitis, P.G., Pema, E.: A dichotomy in the complexity of consistent query answering for queries with two atoms. *Inf. Process. Lett.* **112**(3), 77–85 (2012)
16. Kolaitis, P.G., Pema, E., Tan, W.: Efficient querying of inconsistent databases with binary integer programming. *PVLDB* **6**(6), 397–408 (2013)
17. Koutris, P., Wijsen, J.: The data complexity of consistent query answering for self-join-free conjunctive queries under primary key constraints. In: Proceedings of the 34th ACM Symposium on Principles of Database Systems. PODS 2015, Melbourne, Victoria, Australia, May 31 – June 4, 2015, pp. 17–29 (2015)
18. Marileo, M.C., Bertossi, L.E.: The consistency extractor system: answer set programs for consistent query answering in databases. *Data Knowl. Eng.* **69**(6), 545–572 (2010)
19. Staworko, S., Chomicki, J., Marcinkowski, J.: Prioritized repairing and consistent query answering in relational databases. *Ann. Math. Artif. Intell.* **64**(2–3), 209–246 (2012)
20. Wijsen, J.: Certain conjunctive query answering in first-order logic. *ACM Trans. Database Syst.* **37**(2), 9 (2012)
21. Wijsen, J.: A survey of the data complexity of consistent query answering under key constraints. In: 8th International Symposium on Foundations of Information and Knowledge Systems. FoIKS 2014, Bordeaux, France, March 3–7, 2014. Proceedings, pp. 62–78 (2014)

From SAT to ASP and Back!?

Torsten Schaub^{1,2}

¹ University of Potsdam, Potsdam, Germany

² INRIA Rennes, Rennes, France

torsten@cs.uni-potsdam.de

Answer Set Programming (ASP; [1–4]) provides an approach to declarative problem solving that combines a rich yet simple modeling language with effective Boolean constraint solving capacities. This makes ASP a model, ground, and solve paradigm, in which a problem is expressed as a set of first-order rules, which are subsequently turned into a propositional format by systematically replacing all variables, before finally the models of the resulting propositional rules are computed. ASP is particularly suited for modeling problems in the area of Knowledge Representation and Reasoning involving incomplete, inconsistent, and changing information due to its nonmonotonic semantic foundations. As such, it offers, in addition to satisfiability testing, various reasoning modes, including different forms of model enumeration, intersection or unioning, as well as multi-objective optimization. From a formal perspective, ASP allows for solving all search problems in NP (and NP^{NP}) in a uniform way, that is, by separating problem encodings and instances. Hence, ASP is well-suited for solving hard combinatorial search (and optimization) problems. Interesting applications of ASP include decision support systems for NASA shuttle controllers [5], industrial team-building [6], music composition [7], natural language processing [8], package configuration [9], phylogenetics [10], robotics [11, 12], systems biology [13–15], timetabling [16], and many more. The versatility of ASP is nicely reflected by the ASP solver *clasp* [17], winning first places at various solver competitions, including ASP, MISC, PB, and SAT. In fact, *clasp* is at the heart of the open source platform `potassco.sourceforge.net`. *Potassco* stands for the “Potsdam Answer Set Solving Collection” [18] and has seen more than 145000 downloads world-wide since its inception at the end of 2008.

The talk will start with a gentle introduction to ASP, while focusing on the commonalities and differences to SAT. It will discuss the different semantic foundations and describe the impact of a modelling language along with off-the-shelf grounding systems. Finally, it will highlight some resulting techniques, like meta-programming, preference handling, heuristic constructs, and theory reasoning.

References

1. Gelfond, M., Lifschitz, V.: The stable model semantics for logic programming. In: Kowalski, R., Bowen, K. (eds.) *Proceedings of the Fifth International Conference and Symposium of Logic Programming. ICLP1988*, pp. 1070–1080. MIT Press (1988)

2. Baral, C.: Knowledge Representation, Reasoning and Declarative Problem Solving. Cambridge University Press (2003)
3. Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T.: Answer Set solving in practice. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan and Claypool Publishers (2012)
4. Gelfond, M., Kahl, Y.: Knowledge Representation, Reasoning, and the Design of Intelligent Agents: The Answer-Set Programming Approach. Cambridge University Press (2014)
5. Nogueira, M., Balduccini, M., Gelfond, M., Watson, R., Barry, M.: An A-prolog decision support system for the space shuttle. In: Ramakrishnan, I. (ed.) PADL 2001. LNCS, vol. 1990, pp. 169–183. Springer, Berlin (2001)
6. Grasso, G., Iiritano, S., Leone, N., Lio, V., Ricca, F., Scalise, F.: An ASP-based system for team-building in the Gioia-Tauro seaport. In: Carro, M., Peña, R. (eds.) PADL 2010. LNCS, vol. 5937, pp. 40–42. Springer, Berlin (2010)
7. Boenn, G., Brain, M., de Vos, M., Fitch, J.: Automatic composition of melodic and harmonic music by answer set programming. In: García de la Banda, M., Pontelli, E. (eds.) ICLP 2008. LNCS, vol. 5366, pp. 160–174. Springer, Berlin (2008)
8. Schwitter, R.: The jobs puzzle: taking on the challenge via controlled natural language processing. *Theory Pract. Logic Program.* **13**(4–5), 487–501 (2013)
9. Gebser, M., Kaminski, R., Schaub, T.: aspcud: A Linux package configuration tool based on answer set programming. In: Drescher, C., Lynce, I., Treinen, R., (eds.) Proceedings of the Second International Workshop on Logics for Component Configuration. LoCoCo 2011. Electronic Proceedings in Theoretical Computer Science (EPTCS), vol. 65, pp. 12–25 (2011)
10. Brooks, D., Erdem, E., Erdogan, S., Minett, J., Ringe, D.: Inferring phylogenetic trees using answer set programming. *J. Autom. Reason.* **39**(4), 471–511 (2007)
11. Chen, X., Ji, J., Jiang, J., Jin, G., Wang, F., Xie, J.: Developing high-level cognitive functions for service robots. In: van der Hoek, W., Kaminka, G., Lespérance, Y., Luck, M., Sen, S. (eds.) Proceedings of the Ninth International Conference on Autonomous Agents and Multiagent Systems. AAMAS 2010, IFAAMAS, pp. 989–996 (2010)
12. Erdem, E., Haspalmutgil, K., Palaz, C., Patoglu, V., Uras, T.: Combining high-level causal reasoning with low-level geometric reasoning and motion planning for robotic manipulation. In: Proceedings of the IEEE International Conference on Robotics and Automation. ICRA 2011, pp. 4575–4581. IEEE (2011)
13. Erdem, E., Türe, F.: Efficient haplotype inference with answer set programming. In: Fox, D., Gomes, C. (eds.) Proceedings of the Twenty-Third National Conference on Artificial Intelligence. AAAI 2008, pp. 436–441. AAAI Press (2008)
14. Gebser, M., Schaub, T., Thiele, S., Veber, P.: Detecting inconsistencies in large biological networks with answer set programming. *Theory Pract. Logic Program.* **11**(2–3), 323–360 (2011)
15. Gebser, M., Guziolowski, C., Ivanchev, M., Schaub, T., Siegel, A., Thiele, S., Veber, P.: Repair and prediction (under inconsistency) in large biological networks with answer set programming. In: Lin, F., Sattler, U. (eds.) Proceedings of the Twelfth International Conference on Principles of Knowledge Representation and Reasoning. KR 2010, pp.497–507. AAAI Press (2010)
16. Banbara, M., Soh, T., Tamura, N., Inoue, K., Schaub, T.: Answer set programming as a modeling language for course timetabling. *Theory Pract. Logic Program.* **13**(4–5), 783–798 (2013)
17. Gebser, M., Kaufmann, B., Schaub, T.: Conflict-driven answer set solving: from theory to practice. *Artif. Intell.* **187–188**, 52–89 (2012)
18. Gebser, M., Kaminski, R., Kaufmann, B., Ostrowski, M., Schaub, T., Schneider, M.: Potassco: the Potsdam answer set solving collection. *AI Commun.* **24**(2), 107–124 (2011)

Satisfiability Testing, a Disruptive Technology in Program Verification?

David Monniaux^{1,2}

¹ Université Grenoble Alpes, VERIMAG, 38000 Grenoble, France

² CNRS, Verimag, 38000 Grenoble, France

David.Monniaux@imag.fr

Abstract. In the 2000s, progress in satisfiability testing shook automated and assisted program verification. The advent of efficient satisfiability modulo theory (SMT) solvers allowed new approaches: efficient testing and symbolic execution, new methods for generating inductive invariants, and more automated assisted proof.

Program verification consists in proving properties of software, be them safety (“whatever the program execution, some property is always satisfied”) or liveness (“some action will always eventually happen”). Improvements in satisfiability testing have allowed exciting combinations of exact decision procedures [12], often based on Boolean satisfiability [2], with existing approaches.

1 Program Verification Before SMT

Traditionally, program verification (i) either relied heavily on the user tediously providing *inductive invariants*, *ranking functions* as well as proofs (ii) either simplified the problems through abstractions that sometimes were sufficient to prove the property, sometimes were not. Excessively coarse abstractions (e.g. one single interval of variation per program variable per location) could sometimes be refined by explicit partitioning [16], but its cost is exponential and thus it must be quickly limited.

Program verification was most often, and still is, considered too costly in human and algorithmic terms, and thus in most practical cases, it was replaced by *testing*, with test cases chosen by hand or through test case generation techniques. *Fuzzing* is a kind of testing where variant of input files or protocol exchanges are randomly modified so as to trigger bugs in parsers, which could be exploited as security vulnerabilities.

Both program verification and testing were transformed by the advent of a disruptive technology: *satisfiability modulo theory* (SMT), that is, efficient algorithms for checking that a first-order logic formula over a given theory is satisfiable — e.g. $(x < y \vee x \geq y + 3) \wedge (x \geq 0 \wedge x + y \leq 5)$ is a first-order formula over linear real or integer arithmetic.

The research leading to these results has received funding from the European Research Council under the European Union’s Seventh Framework Programme (FP/2007-2013)/ERC Grant Agreement nr. 306595 “STATOR”.

2 Testing

In *bounded model checking* [1], the program is unrolled up to a finite depth, and a first-order formula is generated, whose solutions are the program traces that violate the desired property before the given depth. In *symbolic execution*, sequences of program statements are translated into a first-order formula, and the feasibility of tests into new branches is checked by satisfiability testing; this approach has been successfully applied to *fuzzing*, that is, searching for inputs that trigger security violations in file or protocol parsers [8]. If symbolic execution is too costly, certain unknowns may be chosen to have concrete values, while others are left symbolic, leading to *concolic execution*.

3 Automatic Verification

Bounded model checking and symbolic execution cannot prove the safety of programs, except in the rare case where there is a small constant bound on execution lengths. Inductive invariant inference was also greatly transformed by the advent of satisfiability modulo theory solvers. First, inference approaches designed to operate over a control-flow graph and produce an invariant per control location were modified to traverse loop-free program fragments (*large blocks*) encoded into first order formulas, whose satisfiability is checked by SMT [7, 15]. Second, *counterexample-guided abstraction refinement* approaches mine proofs of unreachability of errors through finite unrollings for arguments that could become inductive, in particular by extraction of *Craig interpolants* [14].

Research in SMT solving has strived to extend the class of formulas handled by solvers [12]: from quantifier-free linear real arithmetic, solved by a combination of *constraint-driven clause learning* (CDCL) SAT-solving [13] and exact-precision simplex algorithm [5], and *uninterpreted functions*, solvers were extended to linear integer arithmetic, nonlinear (polynomial) arithmetic, arrays, bitvector arithmetic, character strings, data structures, and quantified formulas. Some of these combinations are undecidable or have high lower bounds on their worst-case complexity [6], yet this is not considered a major hindrance; practical efficiency is paramount.

4 Assisted Verification

In assisted proof, using tools such as Coq or Isabelle, the user traditionally has to provide detailed arguments why the claimed theorem is true. Automation is traditionally limited. When the theorem, or a part of it, fits within a class decidable by SMT, it is tempting to check it using SMT. It may however be unwise to blindly trust a SMT solver, a complex piece of software likely to have bugs; and some assistants require all

proofs to be broken down into basic steps checked by a *proof checker*. It is therefore desirable that the solver provides an independently checkable *proof witness*; one challenge is to keep the witness small enough to be manageable while keeping the checker small [3, 11].

5 Challenges

Most extant SMT solvers are based on the DPLL(T) framework: a combination of a CDCL SAT solver and a decision procedure for conjunctions. This framework has known weaknesses: for instance, some industrially relevant families of formulas induce exponential behavior [9]. There have been multiple proposals for solvers based on other approaches [4, 10], but they are less mature. Quantifiers are still often difficult to handle. Verification approaches based on Craig interpolation are often brittle; more generally, reliance on random number generators sometimes results in unpredictable behavior. Better collaboration between verification and SAT experts is needed to overcome these challenges.

References

1. Biere, A.: Bounded model checking. In: Biere et al. [2], vol. 185, pp. 455–481 (2009)
2. Biere, A., Heule, M.J.H., van Maaren, H., Walsh, T. (eds.): Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications, vol. 185. IOS Press (2009)
3. Böhme, S., Weber, T.: Fast LCF-style proof reconstruction for Z3. In: Kaufmann, M., Paulson, L.C. (eds.) ITP 2010. LNCS, vol. 6172, pp. 179–194. Springer, Berlin (2010)
4. Brain, M., D’Silva, V., Griggio, A., Haller, L., Kroening, D.: Deciding floating-point logic with abstract conflict driven clause learning. *Formal Methods Syst. Des.* **45**(2), 213–245 (2014)
5. Dutertre, B., de Moura, L.M.: Integrating simplex with DPLL(T). SRI-csl-06-01, SRI International, Computer Science Laboratory (2006)
6. Fischer, M.J., Rabin, M.O.: Super-exponential complexity of presburger arithmetic. In: Karp, R. (ed.) *Complexity of Computation*. SIAM–AMS Proceedings, pp. 27–42, no. 7. American Mathematical Society (1974). citeseer.ist.psu.edu/fischer74supereponential.html
7. Gawlitza, T., Monniaux, D.: Invariant generation through strategy iteration in succinctly represented control flow graphs. *Logical Methods in Computer Science* (2012)
8. Godefroid, P., Levin, M.Y., Molnar, D.: SAGE: Whitebox fuzzing for security testing. *Queue* **10**(1), 20:20–20:27 (2012)
9. Henry, J., Asavoae, M., Monniaux, D., Maiza, C.: How to compute worst-case execution time by optimization modulo theory and a clever encoding of program semantics. In: Zhang, Y., Kulkarni, P. (eds.) *Languages, Compilers, Tools and Theory for Embedded Systems (LCTES)*, pp. 43–52. ACM (2014)
10. Jovanović, D., de Moura, L.: Solving non-linear arithmetic. In: *IJCAR* (2012)
11. Keller, C.: Extended resolution as certificates for propositional logic. In: Blanchette, J.C., Urban, J. (eds.) *Proof Exchange for Theorem Proving (PxTP)*. EPIc Series, vol. 14, pp. 96–109. EasyChair (2013). <http://www.easychair.org/publications/?page=117514525>

12. Kroening, D., Strichman, O.: *Decision Procedures*. Springer (2008)
13. Marques-Silva, J.P., Lynce, I., Malik, S.: Conflict-driven clause learning SAT solvers. In: Biere et al. [2], vol. 185, pp. 131–153 (2009)
14. McMillan, K.L.: Lazy abstraction with interpolants. In: Ball, T., Jones, R.B. (eds.) *CAV 2006*. LNCS, vol. 4144, pp. 123–136. Springer, Berlin (2006)
15. Monniaux, D., Gonnord, L.: Using bounded model checking to focus fixpoint iterations. In: Yahav, E. (ed.) *SAS 2011*. LNCS, vol. 6887, pp. 369–385. Springer, Berlin (2011)
16. Rival, X., Mauborgne, L.: The trace partitioning abstract domain. *ACM Trans. Program. Lang. Syst.* **29**(5) (2007)

Contents

Complexity

Parameterized Compilation Lower Bounds for Restricted CNF-Formulas	3
<i>Stefan Mengel</i>	
Satisfiability via Smooth Pictures	13
<i>Mateus de Oliveira Oliveira</i>	
Solution-Graphs of Boolean Formulas and Isomorphism	29
<i>Patrick Scharpfenecker and Jacobo Torán</i>	
Strong Backdoors for Default Logic	45
<i>Johannes K. Fichte, Arne Meier, and Irina Schindler</i>	
The Normalized Autocorrelation Length of Random Max r -Sat Converges in Probability to $(1 - 1/2^r)/r$	60
<i>Daniel Berend and Yochai Twitto</i>	
Tight Upper Bound on Splitting by Linear Combinations for Pigeonhole Principle.	77
<i>Vsevolod Oparin</i>	

Satisfiability Solving

Extreme Cases in SAT Problems.	87
<i>Gilles Audemard and Laurent Simon</i>	
Improved Static Symmetry Breaking for SAT.	104
<i>Jo Devriendt, Bart Bogaerts, Maurice Bruynooghe, and Marc Denecker</i>	
Learning Rate Based Branching Heuristic for SAT Solvers.	123
<i>Jia Hui Liang, Vijay Ganesh, Pascal Poupart, and Krzysztof Czarnecki</i>	
On the Hardness of SAT with Community Structure	141
<i>Nathan Mull, Daniel J. Fremont, and Sanjit A. Seshia</i>	
Trade-offs Between Time and Memory in a Tighter Model of CDCL SAT Solvers.	160
<i>Jan Elffers, Jan Johannsen, Massimo Lauria, Thomas Magnard, Jakob Nordström, and Marc Vinyals</i>	

Satisfiability Applications

A SAT Approach to Branchwidth	179
<i>Neha Lodha, Sebastian Ordyniak, and Stefan Szeider</i>	
Computing Maximum Unavoidable Subgraphs Using SAT Solvers	196
<i>C.K. Cuong and M.J.H. Heule</i>	
Heuristic NPN Classification for Large Functions Using AIGs and LEXSAT	212
<i>Mathias Soeken, Alan Mishchenko, Ana Petkovska, Baruch Sterin, Paolo Ienne, Robert K. Brayton, and Giovanni De Micheli</i>	
Solving and Verifying the Boolean Pythagorean Triples Problem via Cube-and-Conquer	228
<i>Marijn J.H. Heule, Oliver Kullmann, and Victor W. Marek</i>	

Satisfiability Modulo Theory

Deciding Bit-Vector Formulas with mcSAT	249
<i>Aleksandar Zeljić, Christoph M. Wintersteiger, and Philipp Rümmer</i>	
Solving Quantified Bit-Vector Formulas Using Binary Decision Diagrams . . .	267
<i>Martin Jonáš and Jan Strejček</i>	
Speeding up the Constraint-Based Method in Difference Logic	284
<i>Lorenzo Candeago, Daniel Larraz, Albert Oliveras, Enric Rodríguez-Carbonell, and Albert Rubio</i>	
Synthesis of Domain Specific CNF Encoders for Bit-Vector Solvers	302
<i>Jeevana Priya Inala, Rohit Singh, and Armando Solar-Lezama</i>	

Beyond SAT

Finding Finite Models in Multi-sorted First-Order Logic	323
<i>Giles Reger, Martin Suda, and Andrei Voronkov</i>	
MCS Extraction with Sublinear Oracle Queries	342
<i>Carlos Mencía, Alexey Ignatiev, Alessandro Previti, and Joao Marques-Silva</i>	
Predicate Elimination for Preprocessing in First-Order Theorem Proving	361
<i>Zurab Khasidashvili and Konstantin Korovin</i>	

Quantified Boolean Formula

Incremental Determinization	375
<i>Markus N. Rabe and Sanjit A. Seshia</i>	
Non-prenex QBF Solving Using Abstraction	393
<i>Leander Tentrup</i>	
On Q-Resolution and CDCL QBF Solving	402
<i>Mikoláš Janota</i>	
On Stronger Calculi for QBFs	419
<i>Uwe Egly</i>	
Q-Resolution with Generalized Axioms	435
<i>Florian Lonsing, Uwe Egly, and Martina Seidl</i>	
2QBF: Challenges and Solutions	453
<i>Valeriy Balabanov, Jie-Hong Roland Jiang, Christoph Scholl, Alan Mishchenko, and Robert K. Brayton</i>	

Dependency QBF

Dependency Schemes for DQBF	473
<i>Ralf Wimmer, Christoph Scholl, Karina Wimmer, and Bernd Becker</i>	
Lifting QBF Resolution Calculi to DQBF	490
<i>Olaf Beyersdorff, Leroy Chew, Renate A. Schmidt, and Martin Suda</i>	
Long Distance Q-Resolution with Dependency Schemes	500
<i>Tomáš Peitl, Friedrich Slivovsky, and Stefan Szeider</i>	

Tools

BEACON: An Efficient SAT-Based Tool for Debugging $\mathcal{EL}^+$ Ontologies . . .	521
<i>M. Fareed Arif, Carlos Mencía, Alexey Ignatiev, Norbert Manthey, Rafael Peñaloza, and Joao Marques-Silva</i>	
HordeQBF: A Modular and Massively Parallel QBF Solver	531
<i>Tomáš Balyo and Florian Lonsing</i>	
LMHS: A SAT-IP Hybrid MaxSAT Solver	539
<i>Paul Saikko, Jeremias Berg, and Matti Järvisalo</i>	
OpenSMT2: An SMT Solver for Multi-core and Cloud Computing	547
<i>Antti E.J. Hyvärinen, Matteo Marescotti, Leonardo Alt, and Natasha Sharygina</i>	

SpyBug: Automated Bug Detection in the Configuration Space
of SAT Solvers. 554
 Norbert Manthey and Marius Lindauer

Author Index 563